

PRACTICE EXAM

Sample Questions

for Claude Architect Exam

Dump 1

66 Questions • Agentic Systems • Context & Escalation • Batch API • Extraction • Tool Design • MCP • AI Fluency

Topic Index & Weightage

TOPIC	Q RANGE	COUNT	WEIGHT
 Context & Escalation (Agentic)	Q1-Q12	12	 18%
 Batch API & Extraction	Q13-Q20	8	 12%
 Tool Design	Q21-Q34	14	 21%
 MCP Design Patterns	Q35-Q46	12	 18%
 Agentic Systems & Code Agents	Q47-Q52	6	 12%
 AI Fluency Framework	Q53-Q66	14	 19%

 Higher-weighted topics = more exam questions. Prioritize Tool Design, MCP, and Agentic Systems for maximum score impact.

SECTION



Context & Escalation

Managing conversation state, tool failures, and human routing

Your agent has called `lookup_order` multiple times while investigating a customer's return requests. Each response includes 40+ fields. Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders. What's the most effective approach before making additional lookups?

A Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response.

✓ CORRECT

B Proceed with additional lookups without modifying the existing tool output context.

C Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues.

D Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions.



Answer A — Selective field extraction reduces tokens while preserving the structured data tools need. Vector DB (C) adds latency; prose summaries (D) lose structured precision. Verbose details.

After investigating a billing dispute over 25+ turns, you've identified duplicate charges due to a payment gateway timeout. The refund (\$847) exceeds your \$500 authorization limit. You call `escalate_to_human`, but the human agent won't have your transcript. What context should you pass?

A The customer's original complaint verbatim plus tool result excerpts showing duplicate transactions.

B A structured summary: customer ID, root cause, refund amount, and recommended action.

✓ CORRECT

C The complete conversation transcript with all tool results.

D Your diagnosis and the refund amount only.



Answer B — A structured summary gives the human exactly what they need without cognitive overload. Raw transcripts (C) are too verbose; diagnosis-only (D) omits customer context.

The agent verifies identity through a multi-step process before resetting passwords. During testing, after the customer answers the third verification question, the agent asks for their name again — as if the earlier exchange never happened. What's the most likely cause?

A Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.

B The verification tool is clearing the agent's internal state after each successful validation step.

C The conversation history isn't being passed in subsequent API requests.

✓ CORRECT

D The prompt lacks instructions telling Claude to remember information across multiple exchanges.

 Answer C — Claude is stateless; it only knows what's in the current request. If history is not included in each API call, prior turns are invisible. There is no default two-turn memory limit (A).

A customer returns 4 hours after their initial 32-turn session. Previous tool results show 'Status: PENDING'. You observe the agent references outdated data even after fresh tool calls return different information. What approach most reliably handles returning customers?

A Resume with full history and configure the agent to automatically re-call all previously-used tools at session start.

B Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.

C Resume with full history but filter out previous tool_result messages.

D Start a new session, inject a structured summary of the previous interaction, then make fresh tool calls before engaging.

✓ CORRECT



Answer D — A fresh session + structured summary + fresh tool calls eliminates stale data conflicts entirely. Keeping old tool results (A, B, C) risks the model incorrectly weighting outdated information.

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

A The order details are added to the conversation and the model reasons about which action to take.

✓ CORRECT

B The agent executes the remaining steps in a tool sequence planned at the start of the request.

C The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.

D The orchestration layer automatically routes to the next tool based on the order's status field.

 Answer A — Tool results are appended to context and the model generates the next action by reasoning over the full conversation. There is no separate routing layer (D).

Production logs reveal inconsistent error handling: `lookup_order` failures trigger 5+ retries, immediate escalation, or user clarification — inconsistently. Your MCP tool returns uniform: `{isError: true, content: [{text: 'Operation failed'}]}`. What's the most effective improvement?

A Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

B Enhance error responses with structured metadata: `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure.

✓ CORRECT

C Add few-shot examples demonstrating how to interpret error message patterns and select appropriate responses — a tempting shortcut that skips the underlying fix.

D Create an `analyze_error` MCP tool the agent calls after any failure to determine the error category and recommended action — an approach that treats the symptom rather than the cause.



Answer B — Structured metadata gives the model unambiguous signals to select the correct recovery strategy. Uniform retries (A) mask non-retriable errors; few-shot parsing (C) is fragile; a second tool call (D) adds unnecessary latency.

Your agent retrieves customer info and order details successfully, but `process_refund` returns a timeout. The agent has enough info to explain charges and verify eligibility but cannot process the refund due to backend failure. What approach best balances first-contact resolution with appropriate error handling?

A Escalate immediately to a human agent — a choice that looks simpler but shifts the problem elsewhere.

B Implement automatic retries with exponential backoff for `process_refund`.

C Explain the billing, confirm refund eligibility, acknowledge the system issue, and offer escalation or retry later.

✓ CORRECT

D Confirm the refund will be processed and close the conversation — an option that holds only under assumptions this scenario breaks.



Answer C — Maximum value: share diagnosis + eligibility, be transparent about the limitation, give customer agency over next steps. False confirmation (D) destroys trust.

Your team proposes four approaches for triggering escalation to human agents. Which approach most reliably identifies cases that genuinely require human intervention?

A Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions.

B Implement sentiment analysis monitoring for frustration indicators and trigger escalation when the score exceeds a configured threshold.

C Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue — a quick fix that tends to resurface the same failure later.

D Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.

✓ CORRECT



Answer D — Semantic conditions (explicit request, policy exception, no progress) capture meaningful escalation reasons. Rules engines (A) miss edge cases; sentiment thresholds (B) produce false positives; fixed retry counts (C) don't reflect actual need.

A customer raises three issues in one session (turns 1–15: refund, 16–30: subscription, 31–45: payment method). At turn 48 they ask 'What happened with my refund?' The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues?

A Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.

✓ CORRECT

B Summarize earlier turns into a narrative description, preserving full message history only for the active issue.

C Implement a sliding window context that retains the most recent 30 turns.

D Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.

 Answer A — Structured extraction preserves key facts from all three issues in a compact, queryable form independent of turn count. Sliding windows (C) discard early issues; re-fetching (D) only works when issues have backend state.

A customer sends: 'This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW.' The agent has not yet called any tools. What should the agent do?

A Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats the request.

B Immediately call `escalate_to_human` with the conversation history.

✓ CORRECT

C First call `get_customer` and `lookup_order` to gather account context, then escalate to a human agent.

D Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.



Answer B — The customer made an explicit, unambiguous request for a human. Honor it immediately. Attempting to deflect (A), gather data first (C), or ask clarifying questions (D) all override a clear customer preference.

Your agent successfully retrieves customer info and order details, but identifies the dispute involves a promotional pricing error requiring manager approval — beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

- A Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- B Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.
- C Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.**
- D Call `escalate_to_human` passing only the customer's original message.

✓ CORRECT



Answer C — A structured handoff ensures the human agent has the full picture without re-investigating from scratch. Passing only the original message (D) discards all diagnostic work.

A customer writes: 'I've been going back and forth on this return for days. I just want to speak to someone who can actually help me.' The agent has confirmed the return is straightforward and eligible for immediate processing. What should the agent do?

- A** Process the refund via `process_refund` to resolve the underlying issue, then inform them it's complete.
- B** Call `escalate_to_human` immediately to honor the customer's request.
- C** Ask what specifically hasn't worked in previous attempts before deciding whether to escalate or resolve automatically.
- D** Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate.

✓ CORRECT



Answer D — The customer expressed frustration and desire for a human, but the root issue is unresolved. B honors autonomy, acknowledges emotion, and surfaces a clear choice — respecting user agency without ignoring the solvable problem.

SECTION



Batch API & Extraction

Cost-effective processing, schema design, quality control

After your daily batch of 10,000 documents completes, 300 (3%) failed with 'context_length_exceeded' errors. The results file identifies each failure by custom_id. What's the most cost-effective approach to process these failures?

A Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions.

✓ CORRECT

B Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch.

C Resubmit the entire 10,000 document batch using a model tier with a larger context window.

D Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts.



Answer A — Only the 300 failures need reprocessing. Chunking resolves context_length_exceeded at minimal cost. Resubmitting all 10,000 (C, D) wastes budget on already-successful documents.

Documents arrive continuously and need extraction within 30 hours of arrival (99.9% SLA). You want to use the Batch API (50% discount, up-to-24-hour window). Which batching strategy is most appropriate?

A

Submit batches every 6 hours containing documents from that window.

B

Submit batches every 4 hours containing documents from that window.

✓ CORRECT

C

Use the real-time API for all documents instead of batch processing.

D

Submit a single batch at end of day containing all documents from that day.



Answer B — 4-hour submission + up-to-24-hour processing = up to 28 hours, within the 30-hour SLA with buffer. 6-hour intervals (A) risks exactly 30 hours with no buffer. End-of-day batching (D) blows the SLA for early arrivals.

After 3 weeks, human reviewers have corrected 847 extractions. 23% of corrections involve informal measurements ('a handful', 'a splash') where the model invents amounts or leaves fields empty. How should you use this feedback?

A Implement a post-processing layer using pattern matching to detect informal measurement phrases and auto-populate empty values.

B Fine-tune the model on the 847 corrected extractions.

C Add few-shot examples demonstrating correct handling of informal measurements.

✓ CORRECT

D Update your JSON schema to add a 'measurement_type' enum field (precise/informal).



Answer C — Few-shot examples directly demonstrate the correct behavior pattern for the identified failure mode. Fine-tuning (B) requires far more data and is premature at 847 samples. Extracting them verbatim rather than converting or omitting them.

After 3 months of 100% human review, extractions with model confidence $\geq 90\%$ show 97% accuracy overall. You plan to automate high-confidence extractions. What validation step is most critical before deploying?

A Compare accuracy at different thresholds (85%, 90%, 95%) to find the optimal cutoff — a workaround that adds complexity without improving reliability.

B Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.

C Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.

D Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

✓ CORRECT



Answer D — Aggregate accuracy can hide catastrophic failure in specific document types or fields. A 97% average that conceals 60% accuracy on one field is dangerous. Segmented analysis must precede any automation decision.

Monitoring shows 12% of extractions fail Pydantic validation with errors like 'expected float for quantity, got 2 to 3'. Retrying without modification produces identical failures. What's the most effective recovery approach?

A Send a follow-up request including the validation error, asking the model to correct its output.

✓ CORRECT

B Set temperature to 0 to eliminate output variability and ensure consistent formatting.

C Implement a secondary pipeline using a larger model tier to reprocess validation failures.

D Pre-process source documents to standardize problematic formats before sending for extraction.

 Answer A — Including the specific validation error in a follow-up gives the model the information it needs to produce a conformant value. Temperature=0 (B) doesn't solve the range format problem.

You process standard monthly reports (archived after processing) and urgent exception reports (must trigger alerts within 30 minutes). Both use the same schema. You want to minimize costs while meeting latency requirements. How should you architect the pipeline?

A Submit all documents to the Batch API. When results arrive, immediately process urgent documents and trigger delayed alerts.

B Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

✓ CORRECT

C Submit all documents to the real-time Messages API to ensure consistent processing latency.

D Queue all documents and submit hourly batches, flagging urgent documents for expedited handling.

 Answer B — Route by latency requirement: Batch API cost savings for non-urgent work, real-time API for the 30-minute SLA. Sending all through Batch (B, D) cannot guarantee the 30-minute window.

12% of extractions contain semantic errors that pass JSON schema validation (e.g., duration placed in quantity field). Reviewers can check only 20% of extractions. Which approach most effectively allocates reviewer attention?

A Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.

B Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types.

C Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set.

✓ CORRECT

D Prioritize review of all extractions where required fields are empty or explicitly marked as not found.



Answer C — Field-level confidence scores calibrated against a labeled set enable targeted, risk-proportional review. Random sampling (A) is fair but inefficient for catching semantic errors.

Your tool_use schema is ~2,500 tokens. Documents under 150K tokens yield 98% accuracy. For documents 175–190K tokens, accuracy drops to 71% with the final third consistently missed. The context window is 200K tokens. Most likely cause?

A

Schemas exceeding 8–10 fields increase decision complexity during parameter generation, reducing accuracy independent of document length — a plausible-sounding move that misreads the root cause.

B

The model distributes attention proportionally across input length, causing fields near the document's end to receive insufficient focus.

C

Very long documents exceed the model's effective attention span regardless of context limits — a route that optimizes effort over correctness here.

D

Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.

✓ CORRECT



Answer D — The 2,500-token schema + system prompt + 175–190K document approaches 200K. Near the context ceiling, the model has less capacity to process end-of-document content. This is a token budget problem.

SECTION



Tool Design

Schemas, errors, UX, pagination, and confirmation patterns

With `tool_choice: 'auto'`, Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You have separate tools for invoices, contracts, and receipts. You need guaranteed structured output without knowing document type in advance. Most effective approach?

- A** Set `tool_choice: 'any'` with all extraction tools defined. ✓ CORRECT
- B** Consolidate all document types into a single unified-schema extraction tool and force that tool.
- C** Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.
- D** Keep `tool_choice: 'auto'` with system prompt instructions requiring tool use.



Answer A — `tool_choice: 'any'` guarantees a tool is called while allowing the model to select the appropriate one. Forced single-tool (B) loses schema specificity; two-call classification (C) adds latency; prompt instructions (D) are not guaranteed.

Your pipeline processes restaurant menus with inconsistent formatting — prices as '\$12' vs '12.00', dietary info as icons vs text. What's the most reliable extraction approach?

A Request multiple extraction attempts per document and select the most common format.

B Extract data as-is and normalize formats in post-processing code after Claude returns.

✓ CORRECT

C Use separate extraction calls for each field to ensure consistent handling.

D Define a strict output schema and include format normalization rules in your prompt.



Answer B — Post-processing code applies deterministic transformation rules reliably at scale. Prompt-based normalization (D) works inconsistently; multiple attempts (A) triple cost; per-field calls (C) explode API usage.

search_catalog fails 12% of the time: 8% are network timeouts (succeed on immediate retry), 4% are query syntax errors (never succeed). Both error types are returned identically, causing wasted retries. How should you modify error handling?

A Apply exponential backoff retry logic to all errors uniformly, returning a generic message after max retries.

B Return all errors with a retryable boolean flag and error type details.

C Implement automatic retry with backoff for network timeouts inside the tool.

✓ CORRECT

D Add few-shot examples demonstrating how to distinguish network errors from syntax errors.

 Answer C — Handle transients transparently inside the tool and surface actionable syntax errors to the agent. Option B is also reasonable but A is more efficient — the agent never wastes turns on transients at all. Return syntax errors immediately with parameter validation details.

A content curation agent makes 18+ sequential tool calls (discover → analyze → add) causing latency. The agent must see all candidates with analysis scores simultaneously to make editorial judgments. What tool composition addresses efficiency while preserving editorial judgment?

A Create a `curate_collection(topic, collection_id)` tool that handles discovery, analysis, and selection internally using configurable quality thresholds.

B Add a `preview_curation` tool with an `approve_curation` tool to confirm.

C Keep all tools separate but implement response caching for `analyze_article` calls.

D Create a `discover_and_analyze(topic)` composite tool returning all candidates with scores.

✓ CORRECT



Answer D — Batching discovery+analysis collapses per-article sequential calls while preserving the model's ability to compare all candidates before deciding what to add. Full composite (A) removes editorial judgment by hardcoding thresholds. `Add_to_collection` separate for selective calls.

Your resource allocation tool returns a simple acknowledgment. Users frequently approve allocations then immediately ask 'how much did that cost?' or 'which project was that?' — indicating approval without understanding. What tool design change is most effective?

A Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response.

✓ CORRECT

B Implement a 60-second hold before execution completes, allowing time to review and cancel — an option that scales poorly as the system grows.

C Add a detail_level parameter ('minimal'/'comprehensive') controlling how much context the agent presents.

D Add a user_acknowledged: boolean parameter the agent sets only after the user confirms they reviewed details.

 Answer A — Providing complete information in the tool response ensures the agent has everything needed for an informed confirmation. The root cause is missing information, not confirmation mechanics.

search_products queries a paginated external API (50 items/page). Queries frequently match 200+ products. Auto-fetching all pages causes 15–20 second delays. How should you redesign pagination?

A Implement server-side relevance ranking and return only the top 50 most relevant items.

B Return the first page with total match count and cursor for additional pages.

✓ CORRECT

C Add a max_pages parameter (default: 2) that controls how many pages are fetched internally.

D Create separate search_products and fetch_more_results tools for pagination.

 Answer B — Returning the first page plus a cursor gives the agent immediate results while enabling progressive fetch only when needed. Max_pages (C) is arbitrary; server-side ranking (A) requires relevance infrastructure.

post_content requires user confirmation before publishing. Users approve 98% within 2 seconds. Post-mortems show posts went to wrong accounts, wrong times, or contained errors — all confirmed without catching mistakes. How should you redesign the confirmation workflow?

A Require users to type a confirmation phrase instead of clicking a button — a change that leaves the real failure mode unaddressed.

B Auto-approve routine posts and only require confirmation for unusual patterns.

C Include the complete post text, target account, scheduled time, and platform in the confirmation request.

✓ CORRECT

D Add a mandatory waiting period before the confirm option becomes available.



Answer C — Users can't catch mistakes they can't see. Surfacing complete post details forces meaningful review. Typing a phrase (A) or delays (D) create friction without adding information; auto-approval (B) removes human oversight.

search_documents needs a parameter to specify which of three databases to search: 'research_papers', 'internal_reports', 'technical_specs'. Users express this naturally in conversation. How should you design the database selection parameter?

A

No explicit parameter — search all three databases by default, then have the model filter results by source — a step that appears safe but reduces observability.

B

A freeform string parameter with runtime validation that returns an error if the value doesn't match a known database.

C

A freeform string parameter where the backend uses semantic matching to determine which database(s) to search.

D

An enum parameter with values ['research_papers', 'internal_reports', 'technical_specs'], requiring the model to map natural language to the appropriate value.

✓ CORRECT



Answer D — Enums constrain the model to valid values and map naturally from conversational intent. The model reliably maps 'research database' → 'research_papers'. Freeform strings (C, D) introduce validation burden.

`publish_article` calls an external CMS API that returns transient errors (timeouts, 503s) and non-transient errors (403, 422). Currently all errors go directly to the agent, causing it to retry non-transient failures. How should you partition error-handling?

- A** Handle transient errors with automatic retries inside the tool. ✓ CORRECT
- B** Handle all errors inside the tool with retries, surfacing failure only after all retries are exhausted.
- C** Surface all errors to the agent immediately with detailed context, letting the agent decide what to retry.
- D** Return a generic 'tool unavailable' message for all exceptions.



Answer A — Correct partition: transients are implementation details the tool handles silently; non-transients (permission denied, validation failure) require agent action and need descriptive context. Surface non-transient errors with descriptive messages so the agent can take corrective action.

Your `track_shipment(tracking_id)` tool raises Python exceptions on errors. Users get unhelpful responses like 'I'm having trouble with that request' instead of actionable suggestions. How should you handle errors in tool results?

A Implement retry logic with exponential backoff inside the tool implementation so transient errors are automatically handled.

B Return structured error information as normal tool output including error type, recoverability status, and actionable context.

✓ CORRECT

C Create dedicated error-recovery tools (`retry_tracking_lookup`, `search_by_order_number`) the model can invoke after failure.

D Return a generic error response (`{success: false, error: 'lookup_failed'}`) for all failure cases to maintain a consistent schema.



Answer B — Rich structured error output gives the model context to suggest actionable alternatives. Python exceptions (A via raising) hide information; generic errors (D) prevent helpful responses.

update_game_score accepts game_date (string), home_team (string), and away_team (string). Logs show the agent uses team nicknames, inconsistent date formats, and selects the wrong game when teams have rematches. Most effective tool interface change?

A Add a season parameter and a confirm_before_update flag returning resolved game details for verification.

B Add detailed examples to the tool description showing required date format and official team names.

C Replace the three parameters with a single game_id parameter and a separate search_games lookup tool.

✓ CORRECT

D Add enum constraints listing valid team names and a regex pattern enforcing ISO 8601 dates.



Answer C — game_id eliminates all three failure modes: no nickname ambiguity, no date format issues, no rematch confusion. Enums (D) help with names/dates but don't solve rematch disambiguation.

Your schema includes a `skills: string[]` field. Production monitoring reveals: compound phrases sometimes split/kept; implied skills occasionally appear; similar documents produce wildly different array lengths (5–10 vs 40+ entries). Current prompt: 'Extract all skills mentioned.' Most effective improvement?

A Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.

B Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

C Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

D Add constraints: 'Extract 10–20 skills maximum, one skill per entry, only explicitly named skills.'

✓ CORRECT



Answer D — Explicit constraints directly address all three failure modes: splitting ambiguity (one per entry), hallucinated skills (explicit only), and length variance (10–20 cap).

Your `property_type` field is an enum: ['house', 'apartment', 'condo', 'townhouse']. After deployment, 8% of extractions fail validation. Investigation reveals listings mention 'studio', 'loft', 'duplex', 'mobile home', 'converted warehouse' — and new types keep appearing. Most effective long-term solution?

A Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.

✓ CORRECT

B Add few-shot examples demonstrating how to map unexpected property types to the closest existing enum value.

C Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.

D Add an 'other' value to your enum with a separate `property_type_detail` string field for specifics.

 Answer A — A free-form field with post-processing normalization handles open-ended vocabularies gracefully and scales without schema changes. Expanding enums (C) is maintenance-heavy; 'other' (D) still misses novel types.

The system routes extractions below 85% confidence to human review. A quarterly audit reveals 12% of high-confidence extractions also contain errors — plausible-but-incorrect values from comparison tables, appendices, and ambiguous phrasing. You need a sustainable strategy to catch errors and measure improvement. Most effective approach?

A Add a verification pass that re-extracts from each high-confidence document, flagging cases where two attempts produce different results.

B Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly.

✓ CORRECT

C Lower the confidence threshold from 85% to 70%, routing a larger volume to human review.

D Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.



Answer B — Stratified random sampling provides a statistically valid error rate baseline, enables trend measurement, and surfaces novel failure patterns. Heuristic rules (D) only catch known patterns. Error rate measurement and novel pattern detection.

SECTION



MCP Design Patterns

Error protocols, tool safety, trust, and session design

Your check_availability MCP tool encounters: (1) missing required user_email parameter, (2) calendar API returns 404 — user not in system, (3) calendar API returns 503 — temporarily unavailable. How should each error be reported per MCP's error handling design?

A Report all three as JSON-RPC protocol errors.

B Report all three as tool results with isError: true.

C Report error 1 as a JSON-RPC protocol error.

✓ CORRECT

D Report errors 1 and 2 as JSON-RPC protocol errors.



Answer C — MCP distinguishes protocol errors (malformed requests, missing params) from execution errors (tool ran, outcome was failure). Error 1 is a protocol error. Errors 2 and 3 occurred during valid execution — they are tool result errors with isError: true. Report errors 2 and 3 as tool results with isError: true.

Your `remove_team_member` tool has `dry_run`: boolean for previewing before execution. The agent bypasses the preview in 15% of cases by calling `dry_run=false` directly. You need every removal preceded by a user-confirmed preview. Most reliable approach?

- A Annotate the tool as requiring confirmation and configure the orchestration layer to prompt before forwarding calls.
- B Add instructions and few-shot examples requiring the agent to always call `dry_run=true` first.
- C Add server-side validation permitting `dry_run=false` only when a matching `dry_run=true` call occurred within 60 seconds.

D Replace with two tools: `preview_remove_member` returns impact details and a single-use token.

✓ CORRECT



Answer D — A cryptographic confirmation token architecturally enforces the preview-before-execute sequence. No matter what the agent does, `execute_remove_member` is mechanically impossible without the token from a prior preview. `Execute_remove_member` requires that token, binding execution to the specific previewed action.

Your marketing agent connects to MCP servers from three vendors. Some tools are annotated `readOnlyHint: true`, others `destructiveHint: true`. Your team proposes auto-skipping confirmation for read-only annotated tools. What should guide this decision?

A

Treat tool annotations as untrusted metadata unless the server is trusted. Base confirmation requirements on your assessment of each vendor's trustworthiness, not on their self-reported annotations.

✓ CORRECT

B

MCP servers run as local processes so tools from any properly initialized connection inherit your application's security context — a tactic that trades long-term reliability for short-term ease.

C

Read-only annotations indicate intended behavior and are reliable for tools that perform GET requests. Skip confirmation for data-fetching tools from properly connected servers.

D

Implement a verification layer that tests each tool's actual behavior before adding it to an allowlist. Skip confirmation only for tools validated through behavioral testing — an easy-to-reach answer that doesn't survive closer scrutiny.



Answer A — MCP annotations are hints set by the server author and cannot be verified. A malicious or misconfigured server could mark destructive tools as read-only. Trust must be based on vendor assessment, not self-reported metadata.

Your productivity agent connects to three MCP servers: an issue tracker, a documentation wiki, and a database explorer. Monitoring shows 8–10 sequential tool calls per cross-system question, frequent exploratory calls because the agent lacks visibility into what content each server contains, and context exhaustion before completing complex investigations. What architectural change best leverages MCP capabilities?

A Add a `prepare_investigation` tool to each server that accepts a natural language question and returns relevant content summaries.

B Expose each server's content catalog as MCP resources.

✓ CORRECT

C Add an orchestrator that routes questions to a single server based on keywords.

D Consolidate all three servers into a unified MCP server with cross-referencing capabilities.

 Answer B — MCP resources are designed for exactly this use case: exposing structured, queryable content catalogs that the agent can read before deciding which tool calls to make, eliminating exploratory calls. Issue summaries, documentation hierarchy, database schemas.

Your codebase exploration tool stores session IDs to allow continuation across work sessions. An engineer analyzed a legacy auth module yesterday, building context about architecture and dependencies. They want to continue today. The session ID is valid, but 3 of the 12 files the agent previously read were modified overnight. What approach best balances efficiency and accuracy?

A Resume the session and immediately have the agent re-read all 12 previously analyzed files.

B Resume the session without informing the agent about the changed files.

C Resume the session and inform the agent which specific files changed for targeted re-analysis.

✓ CORRECT

D Start a fresh session to ensure the agent works with current codebase state without stale assumptions.

 Answer C — Targeted re-analysis of only the 3 changed files preserves the valuable context built over an hour while correcting the specific outdated information. Re-reading all 12 (A) wastes context; ignoring changes (B) risks stale assumptions.

Production logs show the agent calls `delete_file` when users ask to 'remove old backups,' but company policy requires archiving backups. Both tools have minimal descriptions: 'Archives a file' and 'Deletes a file.' Which change most directly improves tool selection?

A

Implement server-side validation that rejects `delete_file` calls for files tagged as backups.

B

Add a confirmation step requiring users to type 'CONFIRM DELETE' before `delete_file` executes.

C

Add few-shot examples showing requests involving 'backup' or 'old' should use `archive_file` — a fix that addresses configuration when the issue is design.

D

Expand tool descriptions to clarify use cases, adding guidance like 'Do not use for backup files' to `delete_file`.

✓ CORRECT



Answer D — Tool descriptions are the primary signal the model uses for tool selection. Adding explicit guidance directly corrects the selection error at the source. Server-side rejection (A) catches the mistake after selection.

15% of booking attempts fail with 'slot no longer available' because another user books between the availability check and the booking call. Tools: `get_available_slots` and `book_appointment`. How should you redesign?

A

Combine both tools into a single `find_and_book_appointment` that atomically checks availability and books, returning confirmed booking or available alternatives.

✓ CORRECT

B

Keep both tools but add retry logic instructing the agent to call `get_available_slots` again if booking fails — an approach that the production logs would later contradict.

C

Modify `book_appointment` to return detailed failure information including currently available alternative slots when the requested slot is unavailable.

D

Add a `hold_slot` tool creating a 60-second temporary reservation between checking and booking — a choice that adds effort without changing the outcome.



Answer A — Atomic check-and-book eliminates the race condition entirely. The 15% failure rate is a concurrency problem solved by removing the gap between check and commit. Retry logic (B) reduces but doesn't eliminate failures.

Your shipment tracking tool queries FedEx (numeric codes), UPS (descriptive phrases), and DHL (timestamped event arrays). The agent uses results to determine delivery status and escalate delays. How should you structure the tool's return value?

A Return both a normalized summary and the complete raw carrier response in each tool call.

B Return a normalized schema with `status`, `estimated_delivery`, `delay_reason`, and `requires_action` fields, converting carrier formats internally.

✓ CORRECT

C Design separate tool endpoints for each carrier with carrier-specific response schemas — a move that confuses tool availability with tool ordering.

D Return raw carrier responses with source metadata, encoding carrier-specific interpretation logic in the system prompt.

 Answer B — A normalized schema lets the agent reason uniformly about delivery status regardless of carrier. Carrier-specific tools (C) force the agent to know which carrier to call; raw responses (D) burden the agent with format translation.

Your system must extract event details from calendar invitations and output JSON conforming to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

A Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.

B Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.

C Define a tool with your target schema as input parameters and have Claude call it with the extracted data.

✓ CORRECT

D Append instructions like 'Output only valid JSON matching the schema exactly' and implement retry logic to re-prompt when JSON parsing fails.



Answer C — Tool use with a typed schema enforces structure at the API level, making malformed output impossible. Prompt instructions (B, C) rely on the model following text directions correctly every time; pre-filling (A) is fragile.

Your system parses product descriptions to extract specifications into JSON. Despite a well-defined schema, the model inconsistently extracts the 'materials' field — sometimes 'cotton blend', other times 'Cotton/Polyester mix', occasionally omitting it when clearly present. Most effective improvement?

- A** Switch to a more capable model tier.
- B** Set temperature to 0 to eliminate randomness and ensure deterministic outputs.
- C** Make the 'materials' field required instead of optional in the schema to force the model to always extract a value.

D Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats.

✓ CORRECT



Answer D — Few-shot examples demonstrate the exact output format expected, directly addressing both the normalization and omission issues. Temperature=0 (B) reduces variance but doesn't fix format inconsistency.

Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

A Implement a hook to intercept tool calls.

✓ CORRECT

B Strengthen the system prompt with emphatic language: 'CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly.'

C Add few-shot examples showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).

D Modify the refund tool to return an error with message 'Amount exceeds policy limit.'

 Answer A — Only a hard-coded programmatic hook provides guaranteed enforcement. Prompt engineering (B, C) and tool errors (D) still rely on model judgment, which has demonstrated a 3% failure rate. When the refund process amount exceeds \$500, block it and invoke human escalation.

`publish_article` calls an external CMS API with transient errors (timeouts, 503s) and non-transient errors (403, 422). Currently all errors go to the agent, causing it to retry non-transient failures. How should you partition error-handling responsibility?

A Handle all errors inside the tool with retries, surfacing failure only after all retries are exhausted.

B Handle transient errors with automatic retries inside the tool.

✓ CORRECT

C Surface all errors to the agent immediately with detailed context, letting the agent decide what to retry.

D Return a generic 'tool unavailable'.



Answer B — Correct partition: transients are implementation details the tool handles silently; non-transients require agent action and need descriptive context. Handling everything internally (A) hides actionable errors. Surface non-transient errors with descriptive messages so the agent can take corrective action.

SECTION



Agentic Systems & Code Agents

Context management, code exploration, and session design

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. The agent has identified that caching logic spans 15 files (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

A

Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.

B

Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.

C

Analyze imports and class hierarchies to identify the base cache class, read that file to understand the interface, then trace specific invalidation implementations.

✓ CORRECT

D

Use Grep to search for 'invalidate' and 'expire' patterns across all files, then read only those specific line ranges with minimal surrounding context.



Answer C — Starting from the base class interface and tracing outward is the most efficient strategy: builds architectural understanding first, then focuses on the specific invalidation pattern needed, minimizing context usage.

Your agent analyzed a complex service module across 23 source files. A developer wants to independently develop two testing strategies (end-to-end vs. snapshot tests) to evaluate trade-offs. How should you manage the sessions?

- A Start two fresh sessions, having each re-read the relevant source files before beginning.
- B Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy.
- C Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- D Export the analysis session's key findings to a file, then create two new sessions that reference this file.**

✓ CORRECT



Answer D — Exporting findings preserves the shared analysis context efficiently, giving both sessions a clean starting point without re-reading 23 files. `fork_session` (B) is not a real Claude feature. Sequential development (C) can't produce independent evaluations.

Your shipment tracking tool queries multiple carriers (FedEx, UPS, DHL) that each return status information in different formats. The agent uses these results to determine delivery status and escalate delayed shipments. How should you structure the tool's return value to best enable the agent's reasoning?

A

Return a normalized schema with `status`, `estimated_delivery`, `delay_reason`, and `requires_action` fields, converting carrier formats internally.

✓ CORRECT

B

Return both a normalized summary and the complete raw carrier response in each tool call — an option that relies on behavior the model can't guarantee.

C

Design separate tool endpoints for each carrier (`track_fedex`, `track_ups`, `track_dhl`) with carrier-specific response schemas.

D

Return raw carrier responses with source metadata, encoding carrier-specific interpretation logic in the system prompt.



Answer A — A normalized schema lets the agent reason uniformly about delivery status regardless of carrier. Carrier-specific tools (C) force the agent to know which carrier to call; raw responses (D) burden the agent with format translation.

Compliance requires refunds exceeding \$500 automatically escalate to a human agent — this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

A

Strengthen the system prompt with emphatic language: 'CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly.'

B

Implement a hook to intercept tool calls.

✓ CORRECT

C

Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).

D

Modify the refund tool to return an error with message 'Amount exceeds policy limit.'



Answer B — Only a hard-coded programmatic hook provides guaranteed enforcement. Prompt engineering (B, C) and tool errors (D) all still rely on model judgment, which has already demonstrated a 3% failure rate. When the refund process amount exceeds \$500, block it and invoke human escalation.

Your billing dispute agent successfully calls `get_customer` and `lookup_order`, but when attempting `process_refund`, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

A Escalate immediately to a human agent — a path that pushes the cost onto a later step instead.

B Implement automatic retries with exponential backoff for `process_refund` — a tactic that works on the happy path but not the edge cases.

C Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later.

✓ CORRECT

D Confirm the refund will be processed and close the conversation — a change that masks the failure rather than resolving it.

 Answer C — Maximum value is delivered by sharing what was accomplished (diagnosis, eligibility), being transparent about the limitation, and giving the customer agency over next steps. False confirmation (D) destroys trust.

You're implementing escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

A Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions.

B Implement sentiment analysis that monitors for frustration indicators and trigger escalation when the frustration score exceeds a configured threshold.

C Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue — an answer that sounds rigorous yet ignores the stated constraint.

D Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.

✓ CORRECT



Answer D — Semantic conditions (explicit request, policy exception, no progress) capture the meaningful reasons for escalation. Rules engines (A) miss edge cases; sentiment thresholds (B) produce false positives; fixed retry counts (C) don't reflect actual need.

SECTION



AI Fluency Framework

The 4Ds model, teaching approaches, and assessment methods

In the linear approach to teaching AI Fluency, students move through the 4Ds in which order?

A Description, Delegation, Diligence, Discernment

✓ CORRECT

B Discernment, Description, Delegation, Diligence

C Delegation, Description, Discernment, Diligence

D Diligence, Delegation, Description, Discernment



Answer A — The linear approach follows the natural progression: Description (defining the task), Delegation (assigning it to AI), Diligence (checking the output), Discernment (making final judgments).

The focused approach to teaching AI Fluency involves:

A Starting with the most advanced concepts

B Deep exploration of a single D at a time

✓ CORRECT

C Teaching all 4Ds simultaneously

D Focusing only on technical skills



Answer B — The focused approach drills deep into one competency at a time before moving to the next, allowing mastery before breadth.

The Delegation-Diligence loop primarily addresses:

A Strategic and ethical decision-making

B Technical implementation

C Tactical, iterative collaboration

✓ CORRECT

D Prompt engineering techniques



Answer C — The Delegation-Diligence loop captures the iterative, practical cycle of assigning tasks to AI and verifying results — the tactical day-to-day collaboration workflow.

The Description-Discernment loop focuses on:

A

Policy development

B

Tactical, iterative collaboration

C

Institutional compliance

D**Strategic and ethical decision-making**

✓ CORRECT



Answer D — The Description-Discernment loop operates at the strategic level: framing what you want (Description) and evaluating whether the outcome aligns with values and goals (Discernment).

The non-linear approach to teaching AI Fluency involves:

A Teaching all 4Ds as an interconnected network

✓ CORRECT

B Deep exploration of a single D at a time

C Starting with the most advanced concepts

D Focusing only on technical skills



Answer A — The non-linear approach treats the 4Ds as an interconnected network, allowing learners to enter at any point and move between competencies fluidly based on context.

Outcome-based assessment primarily focuses on:

A How students think about their thinking

B What students produce through AI collaboration

✓ CORRECT

C Ethical considerations — a choice that improves one metric while degrading another.

D The journey of collaboration



Answer B — Outcome-based assessment evaluates the final products and results students create with AI assistance, measuring what was actually produced.

Process-based assessment examines:

A

Final products only

B

Attendance records

C

How students work with AI over time

✓ CORRECT

D

Test scores — a step that introduces a new dependency for little gain.



Answer C — Process-based assessment focuses on the workflow, decision-making, and iterative collaboration patterns students demonstrate over time — not just the final output.

Reflection-based assessment evaluates:

A

Peer relationships

B

Speed of task completion

C

Technical proficiency

D**Metacognitive awareness**

✓ CORRECT



Answer D — Reflection-based assessment measures metacognitive awareness — how students think about their own thinking and decision-making when collaborating with AI.

Teaching AI Fluency through scenarios can help students understand:

A How competencies connect in practice

✓ CORRECT

B Technical specifications

C Historical context

D Theory only — a tempting shortcut that skips the underlying fix.



Answer A — Scenarios situate the 4Ds in realistic contexts, showing how Description, Delegation, Diligence, and Discernment interact and inform each other in actual work.

In engaging in conversations to integrate the 4Ds into a specific discipline, it is important to ask:

A What does quality look like in our field?

B All of these

✓ CORRECT

C How do we communicate in our discipline?

D What are our field's values and standards?



Answer B — Effective discipline integration requires asking all of these questions: quality standards, communication norms, and values all shape how the 4Ds manifest in a specific field.

As an AI Fluency assignment component, peer review helps students:

A Reduce instructor workload only

B Avoid individual work

C Develop Discernment through evaluating others' work

✓ CORRECT

D Share answers — an approach that treats the symptom rather than the cause.



Answer C — Peer review develops Discernment: the ability to evaluate AI-assisted work critically. Assessing others' outputs sharpens the same judgment students need when evaluating their own AI-generated content.

Which of the following best distinguishes the focused approach from teaching all 4Ds simultaneously?

A Starting with the most advanced concepts

B Focusing only on technical skills

C Teaching all 4Ds simultaneously

D Deep exploration of a single D at a time

✓ CORRECT



Answer D — The focused approach is defined by deep, sequential exploration of one D before advancing to the next, as opposed to parallel or integrated teaching of all four competencies at once.

Which loop primarily addresses tactical, iterative AI collaboration?

A The Delegation-Diligence loop

✓ CORRECT

B The Description-Discernment loop

C The Discernment-Delegation loop

D The Diligence-Description loop



Answer A — The Delegation-Diligence loop captures the tactical back-and-forth of assigning work to AI and checking the output — the iterative, practical cycle of daily AI collaboration.

A course that evaluates students primarily on their AI-collaboration journals, revision histories, and decision logs is using which assessment approach?

A Outcome-based assessment

B **Process-based assessment**

✓ CORRECT

C Reflection-based assessment

D Competency-based assessment



Answer B — Journals, revision histories, and decision logs capture how students work over time — classic process-based assessment artifacts that reveal collaboration patterns rather than just final products.

W R A P - U P

Summary



Context & Escalation 18%

Stateless model, case-facts blocks, lost-in-the-middle, and when to escalate.



Batch API & Extraction 12%

50% cost vs 24h window, custom_id recovery, nullable schemas, retry limits.



Tool Design 21%

Descriptions drive selection; scope per role; typed errors; forced/any tool_choice.



MCP Design Patterns 18%

Project vs user scope, env-var secrets, resources vs tools, hooks for guarantees.



Agentic Systems 12%

Coordinator decomposition, parallel Task spawns, explicit context, crash recovery.



AI Fluency 19%

The 4Ds — Delegation, Description, Discernment, Diligence — and assessment types.

REMEMBER THIS

Key Takeaways



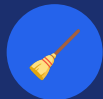
Hard rules need hard enforcement

When a question says "guarantee," "must," or "zero failures," choose programmatic hooks/prerequisite gates — not prompt wording.



Structure your handoffs & errors

Typed error metadata (errorCategory, isRetryable) and compact escalation summaries beat raw transcripts and generic failures.



Practice context hygiene

Trim verbose tool outputs, keep a persistent case-facts block, and counter lost-in-the-middle with summaries + section headers.



Escalate on the right triggers

Explicit human request, policy gap/silence, or no meaningful progress — not sentiment or self-reported confidence.



Tool selection lives in descriptions

Differentiate similar tools, scope each agent to its role, and prefer the proportionate first fix over new ML infrastructure.



Match the API to the latency

Batch API for overnight/latency-tolerant jobs; synchronous API for blocking, interactive, or multi-turn tool-calling work.

BEFORE YOU SIT

Exam Tips

- 1 Read the stem's exact ask** — Distinguish "most effective FIRST step" from "BEST/MOST effective" — the proportionate fix often wins over the maximal one.
- 2 Prefer determinism on "must"** — If a rule must hold every time (money, identity, ordering), pick hooks/gates over prompt instructions, which fail probabilistically.
- 3 Find the real root cause** — In multi-agent failures, blame the agent that made the decision (often the coordinator), not downstream agents working within their scope.
- 4 Reject over-engineering** — When prompt/description tweaks haven't been tried, a new classifier or ML pipeline is usually the wrong, heavier answer.
- 5 Classify errors precisely** — Separate transient vs validation vs business vs permission, and access failures vs valid empty results, before choosing recovery.
- 6 Let stop_reason drive loops** — Continue on "tool_use", stop on "end_turn" — never terminate on text phrases or arbitrary iteration caps.
- 7 Ignore answer length & letter** — Correct options here are randomized in position and length — decide on the concept, not the longest or most-qualified-sounding choice.